

Yes. For this problem, I would separate the variables into **input parameters, LLM interpretation variables, derived variables, optimization decision variables, constraint variables/flags, and output/verification variables.**

The important point is that the official problem is actually quite small mathematically, but there are several **derived variables** you should introduce internally so the optimizer and replay validator stay clean.

1. Scenario / input parameters

These are **given by the request**, not chosen by the optimizer.

A. Time

$H = \{0, 1, \dots, 23\}$
 $h \in H$

Officially the scenario always has exactly 24 hourly entries, hours **0 . . 23**.

B. Hourly energy inputs

For each hour **h**:

demand[h] = demand_kwh
solar[h] = solar_kwh
tariff[h] = tariff_bdt_per_kwh

Meaning:

demand[h] : campus consumption requirement
solar[h] : forecast solar before operator-note adjustments
tariff[h] : grid electricity price

These are exogenous parameters. Your LLM must **not modify them**.

2. Battery input parameters

These are also fixed inputs.

B_max = capacity_kwh
B_init = initial_energy_kwh
B_min = minimum_energy_kwh
C_max = max_charge_kwh_per_hour

`D_max` = `max_discharge_kwh_per_hour`

So conceptually:

`0 <= battery_energy <= B_max`

`battery_energy >= B_min`

`charge <= C_max`

`discharge <= D_max`

These are explicitly part of the request schema.

3. LLM interpretation variables

This group is important because your problem is not just an optimization problem.

For each note `i`:

`note[i]`

`note_index[i]`

`applies[i]`

`directive_type[i]`

`structured_adjustment[i]`

`explanation[i]`

directive_type

Only:

`solar_reduction`

`minimum_battery_reserve`

`no_charge_window`

`no_discharge_window`

`max_grid_window`

`no_op`

are valid.

applies

`applies[i] ∈ {true, false}`

with a very important semantic rule:

no_op -> false
everything else -> true

Structured directive variables

Depending on the type, the LLM can produce:

directive_hours[i]

solar_factor[i]

reserve_requirement[i]

grid_cap[i]

These are essentially the **semantic parameters extracted from language**.

4. Derived directive variables

This is where I would add internal variables even though they are not directly present in the API.

A. Effective solar

This is probably the most important derived variable:

effective_solar[h]

Normally:

$\text{effective_solar}[h] = \text{solar}[h]$

If a solar-reduction directive applies:

$\text{effective_solar}[h] = \text{solar}[h] * \text{factor}$

for the affected hours.

I would make this an explicit array because the rest of the optimizer should operate on **effective_solar**, not repeatedly interpret directives.

B. Active minimum battery reserve

Define:

`active_min_battery[h]`

Normally:

`active_min_battery[h] = B_min`

When a `minimum_battery_reserve` directive applies:

`active_min_battery[h]`
= `max(B_min, directive_reserve[h])`

The specification explicitly says the directive reserve can be higher than the base minimum.

This is a very useful abstraction.

C. Charge allowed flag

Internally:

`charge_allowed[h] ∈ {0,1}`

Normally:

`charge_allowed[h] = 1`

For `no_charge_window`:

`charge_allowed[h] = 0`

Then simply:

`charge[h] ≤ C_max * charge_allowed[h]`

This is much cleaner than repeatedly checking directive arrays.

D. Discharge allowed flag

Similarly:

$\text{discharge_allowed}[h] \in \{0,1\}$

Normally 1.

For `no_discharge_window`:

$\text{discharge_allowed}[h] = 0$

Then:

$\text{discharge}[h] \leq D_{\text{max}} * \text{discharge_allowed}[h]$

E. Grid upper bound

Define:

$\text{grid_max}[h]$

Normally:

$\text{grid_max}[h] = +\infty$

If `max_grid_window` applies:

$\text{grid_max}[h] = \text{specified_max_grid_kwh}$

Then:

$\text{grid}[h] \leq \text{grid_max}[h]$

This lets the optimizer see all operator constraints as simple hourly bounds.

5. Main optimization decision variables

Now we get to the actual mathematical variables.

A. Grid energy

$G[h] = \text{grid_kwh}[h]$

This is how much electricity you purchase from the grid.

Constraint:

$$G[h] \geq 0$$

This is also the variable directly involved in the cost objective.

6. Solar decision variable

$$S[h] = \text{solar_used_kwh}[h]$$

Important distinction:

solar[h] = available forecast

effective_solar[h] = available after directives

S[h] = how much you actually use

Constraint:

$$0 \leq S[h] \leq \text{effective_solar}[h]$$

The rest is simply curtailed. There is **no grid-export variable** in this challenge.

7. Battery decision variables

I strongly recommend separating battery charging and discharging internally, even though the API exposes a single **battery_action**.

Define:

C[h] = battery charge amount

D[h] = battery discharge amount

Both are non-negative:

$$C[h] \geq 0$$

$$D[h] \geq 0$$

Then:

$C[h] \leq C_{\max}$
 $D[h] \leq D_{\max}$

plus:

$C[h] \leq C_{\max} * \text{charge_allowed}[h]$
 $D[h] \leq D_{\max} * \text{discharge_allowed}[h]$

This makes directive enforcement straightforward.

8. Battery state variable

Define:

$E[h]$ = battery energy after hour h

You may also find it useful internally to define:

$E_{\text{before}}[h]$

with:

$E_{\text{before}}[0] = B_{\text{init}}$
 $E_{\text{before}}[h] = E[h-1] \quad \text{for } h > 0$

Then the battery transition becomes extremely clean:

$E[h] = E_{\text{before}}[h] + C[h] - D[h]$

The official rules describe exactly this charge/discharge state transition.

9. Battery state constraints

For every hour:

$\text{active_min_battery}[h] \leq E[h] \leq B_{\max}$

This simultaneously handles:

- normal minimum reserve

- temporary higher reserve directive
 - battery capacity.
-

10. Energy balance constraint

This is the central physical constraint:

$$\begin{aligned} G[h] + S[h] + D[h] \\ = \\ \text{demand}[h] + C[h] \end{aligned}$$

or equivalently:

$$G[h] + S[h] + D[h] - C[h] = \text{demand}[h]$$

This must hold for every hour.

11. End-of-day constraint

This is easy to forget and absolutely critical:

$$E[23] = B_init$$

So the battery must finish with exactly its initial energy.

This means your model is not allowed to simply spend the starting battery permanently.

12. Battery action variable

The API wants:

battery_action[h]

with:

charge

discharge
idle

Internally, I would derive it from $C[h]$ and $D[h]$ rather than make the optimizer directly manipulate strings.

Conceptually:

$C[h] > 0$ and $D[h] = 0 \rightarrow$ charge
 $D[h] > 0$ and $C[h] = 0 \rightarrow$ discharge
 $C[h] = 0$ and $D[h] = 0 \rightarrow$ idle

The response specification requires exactly one action and a zero battery magnitude for idle.

13. A subtle variable you should think about: simultaneous charge/discharge

The spec's external representation has only:

charge
discharge
idle

so your intended model should not allow:

$C[h] > 0$
 $D[h] > 0$

at the same time.

You can enforce that in several ways.

Option 1: binary action variables

$x_charge[h] \in \{0,1\}$
 $x_discharge[h] \in \{0,1\}$
 $x_idle[h] \in \{0,1\}$

with:

$x_charge[h] + x_discharge[h] + x_idle[h] = 1$

and:

```
C[h] <= C_max * x_charge[h]  
D[h] <= D_max * x_discharge[h]
```

This is the safest formulation if you use MILP.

Option 2: structural handling

You can design the solver so that it naturally chooses a single action.

But for a hackathon, **explicit binary action variables are easier to reason about and validate.**

This action encoding is an internal modeling recommendation, not an additional API requirement.

14. Directive-specific constraint variables

Now let's group the variables by operator directive.

solar_reduction

Input/derived:

```
solar_factor[i]  
directive_hours[i]  
effective_solar[h]
```

Constraint:

```
effective_solar[h] = solar[h] * factor
```

for affected hours.

Then:

```
S[h] <= effective_solar[h]
```

minimum_battery_reserve

Variables:

reserve_requirement[h]
active_min_battery[h]

Constraint:

$E[h] \geq \text{active_min_battery}[h]$

where:

$\text{active_min_battery}[h]$
 $= \max(B_{\min}, \text{reserve_requirement}[h])$

no_charge_window

Variable:

charge_allowed[h]

Constraint:

$C[h] = 0$

on affected hours.

Equivalent:

$C[h] \leq 0$

no_discharge_window

Variable:

discharge_allowed[h]

Constraint:

$D[h] = 0$

on affected hours.

max_grid_window

Variable:

grid_max[h]

Constraint:

$G[h] \leq \text{grid_max}[h]$

no_op

No optimization variable.

It should produce:

```
applies = false
directive_type = no_op
structured_adjustment = null
```

and consequently **no constraint modification**.

15. Objective variable

Your optimization objective can simply use:

cost[h]

where:

$\text{cost}[h] = G[h] * \text{tariff}[h]$

Then:

$\text{total_cost} = \sum \text{cost}[h]$

and minimize:

minimize total_cost

The official objective is specifically grid electricity cost.

You do **not** need an objective involving:

solar usage
battery usage
peak demand
battery cycling

unless you choose them as secondary tie-break objectives internally. They are not part of the stated objective.

16. Useful reporting/verification variables

I would maintain these separately from the optimization variables.

total_grid_kwh
total_cost_bdt
peak_grid_kwh

Definitions:

$\text{total_grid_kwh} = \sum G[h]$

$\text{total_cost_bdt} = \sum G[h] * \text{tariff}[h]$

$\text{peak_grid_kwh} = \max(G[h])$

The judge recalculates these from the hourly plan, so don't treat them as independent optimizer decisions.

17. Variables for the LLM guardrail layer

This is outside the mathematical optimizer but still part of the system.

For each note:

raw_note
normalized_note # optional internal
note_index
predicted_directive_type

predicted_hours
predicted_numeric_value
predicted_factor
predicted_reserve
predicted_grid_cap
applies
validation_status
validation_error

For example, you might internally normalize:

"one in the afternoon till three"

into:

[13,14]

but the LLM still needs to be the component interpreting what the note means. The official architecture explicitly separates LLM interpretation from deterministic validation and optimization.

18. Constraint groups I would implement

Instead of thinking of 30 independent constraints, I would organize your solver into these groups:

Group A - Input validity

24 hours
hours = 0..23
 $1 \leq \text{number_of_notes} \leq 3$
valid numeric values

Group B - Solar

$0 \leq S[h]$
 $S[h] \leq \text{effective_solar}[h]$

Group C - Grid

$G[h] \geq 0$
 $G[h] \leq \text{grid_max}[h]$

Group D - Battery state

$\text{active_min_battery}[h] \leq E[h]$
 $E[h] \leq B_{\text{max}}$

Group E - Battery transition

$E[h] = E_{\text{before}}[h] + C[h] - D[h]$

Group F - Battery rates

$C[h] \leq C_{\text{max}}$
 $D[h] \leq D_{\text{max}}$

Group G - Operator restrictions

$\text{no_charge} \rightarrow C[h] = 0$
 $\text{no_discharge} \rightarrow D[h] = 0$
 $\text{reserve} \rightarrow E[h] \geq \text{reserve}$
 $\text{grid cap} \rightarrow G[h] \leq \text{cap}$
 $\text{solar reduction} \rightarrow S[h] \leq \text{solar} * \text{factor}$

Group H - Physical energy balance

$G[h] + S[h] + D[h]$
 $=$
 $\text{demand}[h] + C[h]$

Group I - End-of-day

$E[23] = B_{\text{init}}$

Group J - Action consistency

exactly one action
 $\text{idle} \rightarrow \text{battery_kwh} = 0$

19. "Hidden" variables worth considering

This is where I would be careful.

The challenge says hidden tests can vary:

- demand
- solar
- tariff

- battery state
- reserve/rate limits
- directive combinations

and can paraphrase the same supported directives. It explicitly says hidden cases **do not introduce unpublished directive types**.

So there aren't secret energy variables you need to magically discover.

However, these **internal derived variables** are worth having because they protect against hidden combinations:

effective_solar[h]
 active_min_battery[h]
 charge_allowed[h]
 discharge_allowed[h]
 grid_max[h]
 E_before[h]
 E_after[h]
 C[h]
 D[h]
 G[h]
 S[h]
 cost[h]

Especially these three:

effective_solar[h]
 active_min_battery[h]
 grid_max[h]

because multiple directives can modify different aspects of the same hour.

20. Things you should NOT invent

This is equally important.

The specification does **not** give you parameters/constraints for:

battery charging efficiency
 battery discharging efficiency
 battery self-discharge

solar efficiency
battery degradation
battery cycle cost
grid export revenue
maximum solar curtailment
time-varying battery efficiency
carbon emissions
grid blackout
minimum grid purchase
ramp rate for grid
ramp rate for battery

You should **not introduce these as physical assumptions** because the problem explicitly says the LLM must not invent demand, solar, tariff, battery limits, or unsupported directive behavior.

In particular:

$\text{solar_used} \leq \text{effective_solar}$

is enough. There is no export decision.

And:

battery energy

changes directly by charge/discharge amount. No efficiency term is specified.

21. The complete internal variable picture

A clean implementation could therefore look like this:

INPUT PARAMETERS

h
demand[h]
solar[h]
tariff[h]

B_max
B_init
B_min

C_max
D_max

LLM / DIRECTIVE VARIABLES

note[i]
applies[i]
directive_type[i]
directive_hours[i]

solar_factor[i]
reserve_requirement[i]
grid_cap[i]

DERIVED PARAMETERS

effective_solar[h]
active_min_battery[h]
charge_allowed[h]
discharge_allowed[h]
grid_max[h]

OPTIMIZATION VARIABLES

G[h] grid energy
S[h] solar used
C[h] battery charge
D[h] battery discharge
E[h] battery energy after hour

x_charge[h]
x_discharge[h]
x_idle[h]

cost[h]

GLOBAL / REPORT VARIABLES

total_grid_kwh
total_cost_bdt

peak_grid_kwh

That is the variable set I would start with.

Most important insight

The problem can actually be reduced to a very clean mathematical core:

For $h = 0..23$:

minimize $\sum G[h] * tariff[h]$

subject to:

$G[h] + S[h] + D[h] = demand[h] + C[h]$

$0 \leq S[h] \leq effective_solar[h]$

$0 \leq C[h] \leq C_max$

$0 \leq D[h] \leq D_max$

$active_min_battery[h] \leq E[h] \leq B_max$

$E[h] = E[h-1] + C[h] - D[h]$

$C[h] = 0$ when charging is forbidden

$D[h] = 0$ when discharging is forbidden

$G[h] \leq cap[h]$ when grid cap applies

$E[23] = B_init$

Everything difficult about the hackathon is essentially **how you correctly derive `effective_solar`, `active_min_battery`, `charge_allowed`, `discharge_allowed`, and `grid_max` from messy natural-language notes**. The organizer's hidden tests are specifically designed around that interpretation/application pipeline.

A useful next step would be to turn this into an **actual MILP/LP formulation with exact equations and binary variables**, including how to handle multiple directives on overlapping hours.